

Programación 2

TFI - P2

Trabajo Final Integrador

Egea, Rodrigo

arodrigoegea@gmail.com

Jacinto, Gaston

gastonjacintocontacto@gmail.com

Dominguez, Ivan

ivan_dominguez2016@hotmail.com

Pérez Campos, Joaquín

joaco.ltz@gmail.com

Profesor: Ferro, Tomás

17 de Noviembre de 2025

Índice

1. Dominio seleccionado	2
1.1. Diagrama UML	2
1.2. Clase Producto (A) — Mapeada a la tabla <code>product</code>	2
1.3. Clase CodigoBarras (B) — Mapeada a la tabla <code>barcode</code>	3
1.4. Clase BarcodeType (C) — Mapeada a la tabla <code>barcode_type</code>	4
1.5. Relaciones del modelo	5
2. Arquitectura por Capas del Proyecto	5
2.1. Capa de Presentación (paquete <code>app</code>)	5
2.2. Capa de Servicios (paquete <code>service</code>)	6
2.3. Capa de Persistencia (paquete <code>dao</code>)	6
2.4. Capa de Dominio (paquete <code>entities</code>)	7
2.5. Configuración (paquete <code>config</code>)	7
3. Persistencia, Transacciones y Orden de Operaciones	8
3.1. Estructura de la Base de Datos	8
3.2. Transacciones	8
3.3. Orden Lógico de Operaciones	9
4. Validaciones y Reglas de Negocio	9
4.1. Validaciones en Base de Datos	9
4.2. Validaciones en la Capa de Presentación (<code>AppMenu</code>)	9
4.3. Justificación del Enfoque Adoptado	10
5. Pruebas Realizadas	10
5.1. Capturas del Menú	11
6. Conclusiones y Mejoras Futuras	13
6.1. Mejoras Futuras	14
7. Fuentes y Herramientas Utilizadas	14
8. Enlaces importantes	14

1. Dominio seleccionado

Para el Trabajo Práctico Integrador seleccionamos el dominio **Producto** → **Código de Barras**, complementado con una tercera entidad adicional llamada **Tipo de Código de Barras** (barcode_type). Este dominio resulta adecuado porque representa un caso de uso real y frecuente en sistemas comerciales, logísticos y de inventario, permitiendo modelar relaciones, restricciones y reglas de negocio relevantes.

1.1. Diagrama UML

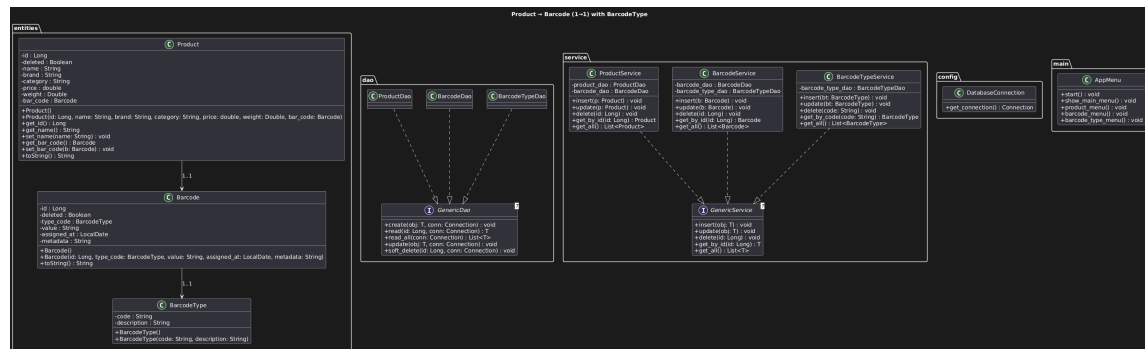


Figura 1: Diagrama UML.

El modelo final presenta una clara separación entre:

- la **información comercial del producto**,
- la **identificación única del producto** mediante un código de barras,
- y la **clasificación del tipo de código de barras** (EAN13, EAN8, UPC).

Por este motivo decidimos incorporar una tercera tabla, **barcode_type**, que normaliza los tipos y mejora la integridad referencial.

1.2. Clase Producto (A) — Mapeada a la tabla product

El esquema SQL real es:

```
CREATE TABLE product (  
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    deleted TINYINT(1) NOT NULL DEFAULT 0,  
    name VARCHAR(120) NOT NULL,  
    brand VARCHAR(80) NULL,  
    category VARCHAR(80) NULL,  
    price DECIMAL(10,2) NOT NULL,  
    weight DECIMAL(10,3) NULL,  
    bar_code_id BIGINT UNSIGNED NULL,  
    PRIMARY KEY (id),  
    UNIQUE KEY uk_product_bar_code_id (bar_code_id),  
    FOREIGN KEY (bar_code_id) REFERENCES barcode(id)  
);
```

Los atributos correspondientes en la clase **Product** y su mapeo son:

- **id** (id en SQL): Long. PK autoincremental.
- **deleted** (deleted): Boolean. Baja lógica.
- **name** (name): String. Obligatorio, máx. 120.
- **brand** (brand): String. Opcional, máx. 80.
- **category** (category): String. Opcional, máx. 80.
- **price** (price): double. Obligatorio, precisión (10,2).
- **weight** (weight): Double. Opcional, precisión (10,3).
- **barCodeId** (bar_code_id): Long. FK hacia barcode(id).

La relación es **1→1 opcional**, ya que un producto puede existir sin código de barras asignado.

1.3. Clase CodigoBarras (B) — Mapeada a la tabla barcode

Estructura SQL real:

```
CREATE TABLE barcode (  
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
    deleted TINYINT(1) NOT NULL DEFAULT 0,  
    type_code VARCHAR(10) NOT NULL,  
    value VARCHAR(20) NOT NULL,  
    assigned_at DATE NULL,  
    metadata VARCHAR(255) NULL,  
    PRIMARY KEY (id),  
    FOREIGN KEY (type_code) REFERENCES barcode_type(code),  
    UNIQUE KEY uk_barcode_value (value)  
);
```

El mapeo a la clase **Barcode** es el siguiente:

- **id** (id): Long. PK autoincremental.
- **deleted** (deleted): Boolean. Baja lógica.
- **typeCode** (type_code): String. FK a barcode_type(code).
- **value** (value): String. Obligatorio, único, máx. 20.
- **assignedAt** (assigned_at): LocalDate. Fecha opcional.
- **metadata** (metadata): String. Máx. 255, opcional.

El atributo `type_code` asegura que todos los códigos de barras pertenecen a un tipo válido definido en la tabla `barcode_type`.

1.4. Clase **BarcodeType** (C) — Mapeada a la tabla `barcode_type`

Dada la necesidad de normalizar los tipos de código y evitar inconsistencias, agregamos explícitamente la siguiente tabla:

```
CREATE TABLE barcode_type (  
    code VARCHAR(10) NOT NULL,  
    description VARCHAR(50) NOT NULL,  
    PRIMARY KEY (code)  
);
```

La clase **BarcodeType** representa dicha tabla y contiene:

- **code** (code): String. PK.
- **description** (description): String. Descripción del estándar (máx. 50).

Esta tabla permite manejar un catálogo controlado de tipos válidos (por ejemplo, *EAN13*, *EAN8*, *UPC*).

1.5. Relaciones del modelo

La relación principal del dominio es:

- **Producto 1→1 Código de Barras**: cada producto puede estar asociado a un único código de barras, garantizado mediante la restricción UNIQUE (`uk_product_bar_code_id`).
- **Código de Barras N→1 Tipo de Código**: múltiples códigos pueden compartir el mismo tipo (*EAN8*, *EAN13*, *UPC*).

La estructura completa constituye un modelo altamente consistente y fiel a los requisitos empresariales típicos de gestión de productos, identificación y validación de códigos de barras. La separación en tres tablas respeta los principios de normalización y facilita la extensibilidad del sistema.

2. Arquitectura por Capas del Proyecto

El sistema fue desarrollado utilizando una arquitectura clásica en capas. Esta organización permite separar responsabilidades, facilitar la mantenibilidad del código y promover un diseño modular. Las capas y sus paquetes asociados corresponden exactamente a la estructura del proyecto implementado.

2.1. Capa de Presentación (paquete app)

La capa de presentación está compuesta por las clases ubicadas en el paquete `app`. Esta capa incluye:

- **Main**: clase principal que inicia la ejecución del sistema.
- **AppMenu**: implementa la interfaz de usuario mediante menús en consola.

Sus responsabilidades son:

- mostrar menús y opciones al usuario,
- capturar entradas desde consola,
- delegar las operaciones a la capa de servicios,
- mostrar los resultados o errores correspondientes.

Es importante destacar que esta capa **no contiene lógica de negocio ni accede directamente a la base de datos**. Toda operación compleja es derivada hacia los servicios.

2.2. Capa de Servicios (paquete service)

La capa de servicios actúa como intermediaria entre la presentación y la capa de persistencia. Incluye:

- ProductService
- BarcodeService
- BarcodeTypeService

Sus responsabilidades principales son:

- gestionar conexiones JDBC mediante `DatabaseConnection`,
- controlar transacciones (commit y rollback),
- aplicar validaciones y reglas de negocio previas a las operaciones,
- delegar las operaciones CRUD a los DAOs correspondientes.

2.3. Capa de Persistencia (paquete dao)

En esta capa se implementa el acceso directo a la base de datos utilizando JDBC. Aquí se encuentran:

- ProductDao
- BarcodeDao
- BarcodeTypeDao

- `GenericDao<T>` y `StringKeyDao<T>` (interfaces base)

Las responsabilidades de esta capa son:

- ejecutar consultas SQL (INSERT, SELECT, UPDATE, DELETE),
- mapear resultados de consultas hacia objetos de entidad,
- no manejar transacciones (responsabilidad exclusiva de la capa de servicios),
- no contener lógica de negocio.

2.4. Capa de Dominio (paquete entities)

Esta capa contiene las clases del modelo de dominio:

- `Product`
- `Barcode`
- `BarcodeType`

Las entidades representan directamente las tablas del esquema SQL. Incluyen únicamente atributos, constructores y métodos de acceso (getters/setters), sin ninguna lógica adicional.

2.5. Configuración (paquete config)

En este paquete se encuentra la clase `DatabaseConnection`, encargada de:

- proporcionar conexiones JDBC listas para usar,
- centralizar la URL, usuario y contraseña,
- permitir separar la lógica de acceso a la base de datos del resto del proyecto.

3. Persistencia, Transacciones y Orden de Operaciones

3.1. Estructura de la Base de Datos

El sistema está compuesto por tres tablas principales:

- **product**: contiene los productos del catálogo.
- **barcode**: modela códigos de barras asociados a productos.
- **barcode_type**: lista de tipos válidos de código de barras (EAN13, EAN8, UPC).

Las claves foráneas garantizan que no existan códigos de barras con tipos inválidos y que un producto solo pueda tener un único código de barras asignado.

3.2. Transacciones

La gestión de transacciones se realiza exclusivamente en la capa de servicios (**service**). El orden típico es:

1. Obtener conexión mediante **DatabaseConnection**.
2. Desactivar **autoCommit**.
3. Ejecutar una o más operaciones DAO.
4. En caso de éxito: **commit**.
5. En caso de error: **rollback**.
6. Cerrar la conexión (uso de **try-with-resources**).

Ejemplo aplicado a **ProductService**:

```
try (Connection conn = DatabaseConnection.getConnection()) {  
    conn.setAutoCommit(false);  
    productDao.update(p, conn);  
    conn.commit();  
} catch (Exception e) {  
    conn.rollback();  
}
```

3.3. Orden Lógico de Operaciones

- Primero deben existir registros en `barcode_type`.
- Luego pueden crearse códigos de barras.
- Finalmente, los productos pueden asociarse a códigos de barras válidos.

4. Validaciones y Reglas de Negocio

El sistema implementa validaciones y reglas de negocio en dos niveles complementarios: por un lado, a nivel de base de datos mediante restricciones SQL; y por otro lado, en la capa de presentación mediante el `AppMenu`.

4.1. Validaciones en Base de Datos

El esquema SQL contiene restricciones que garantizan la integridad mínima del modelo:

- `deleted` toma únicamente los valores 0 o 1 (baja lógica).
- `price > 0`.
- `weight >= 0` o `NULL`.
- `value` en `barcode` es único (no existen dos códigos iguales).
- `bar_code_id` en `product` es único, asegurando la relación $1 \rightarrow 1$.

Estas restricciones permiten descartar datos imposibles o inconsistentes antes de que lleguen al sistema.

4.2. Validaciones en la Capa de Presentación (`AppMenu`)

Las validaciones dinámicas del sistema se implementan en la clase `AppMenu`, donde se verifica:

- que un producto exista antes de actualizarlo o eliminarlo,
- que exista un código de barras antes de asignarlo o modificarlo,

- que no se ingresen precios negativos ni valores fuera de rango,
- que no se intenten eliminar registros inexistentes,
- que la entrada del usuario sea válida antes de delegar al servicio.

4.3. Justificación del Enfoque Adoptado

Si bien en arquitecturas más complejas estas validaciones suelen ubicarse en la capa de servicios, en este proyecto se decidió implementarlas en el **AppMenu** por las siguientes razones:

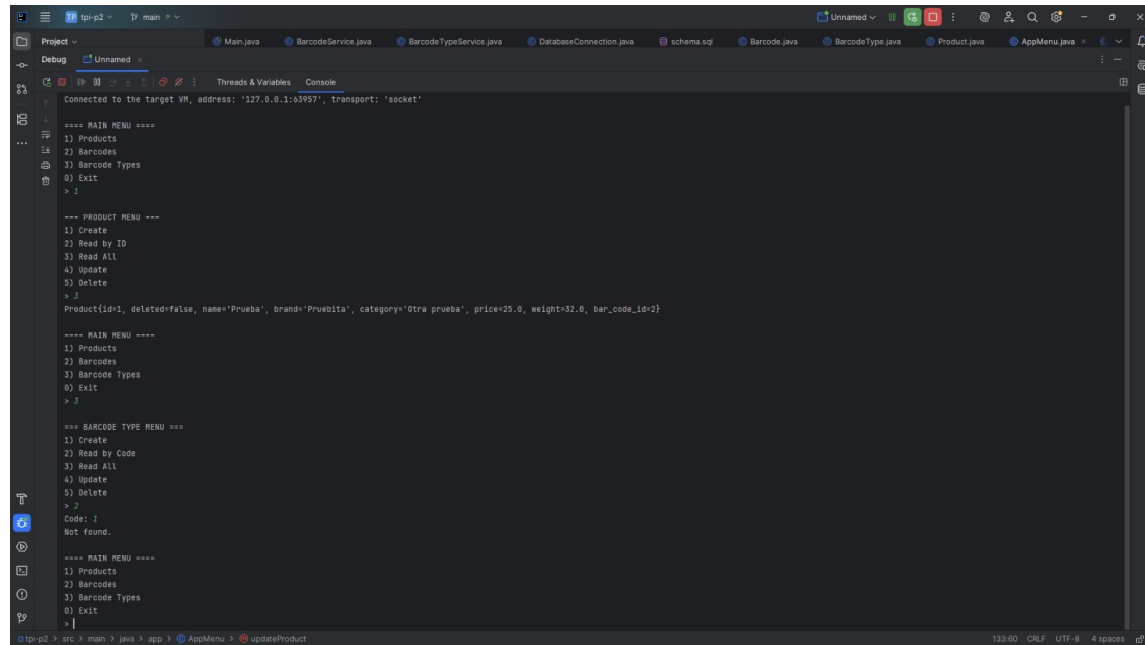
- **Simplicidad del alcance:** el objetivo del trabajo es demostrar dominio de JDBC, DAOs y entidades sin agregar complejidad innecesaria a la capa de servicios.
- **Interacción directa con el usuario:** al validar en la interfaz de consola, se pueden detectar y corregir errores de ingreso antes de ejecutar operaciones costosas sobre la base de datos.
- **Claridad pedagógica:** centralizar las validaciones en el lugar donde se reciben los datos facilita la comprensión del flujo del sistema.
- **Separación de responsabilidades razonable para un proyecto de consola:** la capa de servicios se mantiene enfocada exclusivamente en:
 - manejo de conexiones,
 - control de transacciones (commit / rollback),
 - delegación hacia los DAOs.

5. Pruebas Realizadas

Durante el desarrollo se realizaron pruebas manuales mediante interacción con el menú principal en consola. Se validaron los siguientes casos:

- Creación, lectura, actualización y eliminación lógica de productos.
- Creación y lectura de códigos de barras.
- Asociación de códigos de barras a productos.
- Validación de errores comunes (IDs inexistentes, tipos inválidos, valores negativos).

5.1. Capturas del Menú



```
Connected to the target VM, address: '127.0.0.1:83957', transport: 'socket'

===== MAIN MENU =====
1) Products
2) Barcodes
3) Barcode Types
0) Exit
> 1

===== PRODUCT MENU =====
1) Create
2) Read by ID
3) Read All
4) Update
5) Delete
> 3
Product{id=1, deleted=false, name='Prueba', brand='Prueba', category='Otra prueba', price=25.0, weight=32.0, bar_code_id=2}

===== MAIN MENU =====
1) Products
2) Barcodes
3) Barcode Types
0) Exit
> 2

===== BARCODE TYPE MENU =====
1) Create
2) Read by Code
3) Read All
4) Update
5) Delete
> 3
Code: 1
Not found.

===== MAIN MENU =====
1) Products
2) Barcodes
3) Barcode Types
0) Exit
> 1
```

Figura 2: Operaciones GET ALL y GET BY CODE sobre la tabla 'products' y 'barcode_type', respectivamente.

```
==== MAIN MENU ====
1) Products
2) Barcodes
3) Barcode Types
0) Exit
> 3

=== BARCODE TYPE MENU ===
1) Create
2) Read by Code
3) Read All
4) Update
5) Delete
> 3
BarcodeType{code='EAN13', description='European Article Number 13'}
BarcodeType{code='EAN8', description='European Article Number 8'}
BarcodeType{code='UPC', description='Universal Product Code'}

==== MAIN MENU ====
1) Products
2) Barcodes
3) Barcode Types
0) Exit
> 3

=== BARCODE TYPE MENU ===
1) Create
2) Read by Code
3) Read All
4) Update
5) Delete
> 1
Code: EAN54
Description: Prueba
Barcode Type created.
```

Figura 3: Operación GET ALL sobre la tabla 'barcode_type', que muestra 3 elementos, y luego se crea un nuevo.

```
==== MAIN MENU ====
1) Products
2) Barcodes
3) Barcode Types
0) Exit
> 3

=== BARCODE TYPE MENU ===
1) Create
2) Read by Code
3) Read All
4) Update
5) Delete
> 3

BarcodeType{code='EAN13', description='European Article Number 13'}
BarcodeType{code='EAN54', description='Prueba'}
BarcodeType{code='EAN8', description='European Article Number 8'}
BarcodeType{code='UPC', description='Universal Product Code'}
```

Figura 4: Operación GET ALL sobre la tabla 'barcode_type', para corroborar que se creó el nuevo BarcodeType de la imagen anterior.

6. Conclusiones y Mejoras Futuras

El proyecto permitió aplicar de forma integrada los conceptos de:

- programación orientada a objetos,
- arquitectura por capas,
- acceso a base de datos mediante JDBC,
- diseño de DAOs,
- manejo de transacciones,

- y modelado de entidades y relaciones.

El sistema cumple con las operaciones básicas requeridas, es modular y puede ampliarse fácilmente.

6.1. Mejoras Futuras

- Incorporar una capa de validación más robusta.
- Usar variables de entorno para la conexión con la base de datos.
- Reemplazar la consola por una interfaz web o de escritorio.

7. Fuentes y Herramientas Utilizadas

- MySQL Server 8.0 y MySQL Workbench.
- IntelliJ IDEA Community Edition.
- JDBC Driver `mysql-connector-j`.
- Git y GitHub para control de versiones.
- Java 21.
- PlantUML para diagramas.
- $\text{\LaTeX}$ para la documentación del informe.
- ChatGPT (OpenAI) para asistencia en redacción, depuración y generación de estructuras de código.

8. Enlaces importantes

- Repositorio de GitHub: <https://github.com/GastonJacinto/tpi-p2>
- Video grabado por el equipo: <https://www.youtube.com/watch?v=V1sZHNbqP4Y>